

Git 使用规范与协作指南 (基于 GitLab Flow)

1. 核心分支策略

我们的工作流以“上游优先 (Upstream First)”为核心原则，即所有的新功能和缺陷修复必须首先合并入主干，再向下游的分支（如发布分支、定制分支）流转。

分支类型	命名规范	描述与生命周期	权限与合并策略
主干分支	main (或 master)	唯一的长期演进分支。 代表下一个待发布版本的最新代码， 始终保持可编译、可测试的状态。	严格保护 ：禁止直接 Push。必须通过 MR/PR 合并，需流水线通过及 Code Review。
开发/修复分支	feat/<功能名> fix/<缺陷名>	开发者日常工作分支。从 main 切出， 生命周期极短（建议 1~3 天）。 合并后立即删除。	开发者自由创建与删除。
标准发布分支	stable/v<版本号> 例： stable/v2.0	商业软件对外发版时从 main 切出。 不再接受新功能，只接受从 main 摘取 （Cherry-pick）的 Bug 修复。	保护分支 ：仅限 Release Manager 或核心成员操作。
大客户定制分支	custom/<客户名>/v<版本号> 例： custom/corpA/v2.0	针对特定大客户的私有化部署或特殊定制逻辑。 仅包含无法合入通用主干的定制化代码。	保护分支 ： 长生命周期分支。通用 Bug 修复需从 main 摘取， 严禁反向污染主干。

大客户定制分支维护准则：

坚决杜绝在 custom/<客户名>/... 分支上直接开发通用功能。如果大客户提出了一个既有定制需求，又包含通用 Bug 修复的需求，必须将通用 Bug 修复提交至 main，然后再 Cherry-pick 到定制分支中。保持定制分支与主干的差异最小化。

注意 (新手避坑)：

1. **切分支前先同步**：每次基于 main 切新分支前，必须先执行 `git pull` 获取远端最新代码，否则新分支将基于落后的历史节点开发。

2. **避免超长生命周期分支**：不要让一个 feat 分支在本地存活数周。分支存活越久，最后合并时的冲突灾难就越严重。大需求应拆分为 1~3 天可提交的小任务。

2. Commit 提交规范与详细定义

统一的提交信息不仅让历史记录清晰可读，还能通过脚本自动生成高质量的发布日志 (Release Notes)。采用标准的 Angular 规范格式：

<type>(<scope>): <subject>

2.1 Type (类型) 的严格定义与适用场景

正确选择 Type 是规范化提交的第一步，严禁乱用或张冠李戴：

- **feat (新增功能)**
 - **定义：**引入了全新的业务功能、面向用户的新特性、或对外部开放了新的 API 接口。
 - **示例：** feat(autocad): 新增图纸导出为自定义 XML 格式的接口
- **fix (修复缺陷)**
 - **定义：**修复了系统崩溃、逻辑错误、UI 渲染异常等不符合预期的 Bug。
 - **示例：** fix(wpf): 修复多显示器下弹窗位置偏移导致的闪退问题
- **style (代码格式修改)**
 - **定义：**仅仅是代码格式化的改动（如调整缩进、增删空格、补齐分号、移除未使用的 using 命名空间）。绝对不包含任何业务逻辑的修改，编译后的 IL 代码应与修改前完全一致。
 - **示例：** style: 格式化工程，移除所有文件中未使用的 using 引用
- **refactor (代码重构)**
 - **定义：**对现有代码结构进行重组。既不是新增功能，也不是修改 Bug。其目的是提高代码的可读性、可维护性或降低耦合度（例如提取公共方法、重命名核心变量、引入设计模式）。外部调用方不应感知到行为变化。
 - **示例：** refactor(geometry): 提取多边形求交的底层算法为独立的公共服务类
- **perf (优化相关)**
 - **定义：**针对系统性能或用户体验的专门优化。例如降低内存占用、减少 CPU 消耗、提升计算速度、开启 UI 虚拟化以减少卡顿。
 - **示例：** perf(tekla): 优化节点遍历算法，使超大模型下的读取速度提升 30%
- **test (测试用例)**
 - **定义：**增加新的测试用例，或修改现有测试代码（如单元测试、集成测试）。不涉及任何生产环境代码（业务代码）的变动。
 - **示例：** test(math): 补充针对矩阵变换核心逻辑的单元测试覆盖率
- **chore (工程配置与杂项)**
 - **定义：**构建过程、辅助工具或配置文件的变动。例如修改 MSBuild/CMake 脚本、升级 NuGet/NPM 依赖包、调整 CI/CD 流水线配置文件等。
 - **示例：** chore(msbuild): 迁移传统 .NET Framework 项目至 SDK-style 格式并升级依赖
- **docs (文档更新)**
 - **定义：**仅仅修改了文档（如 [README.md](#)、用户手册）或完善了代码中的 XML 注释。
 - **示例：** docs: 更新二次开发环境搭建的新手指导文档

2.2 Scope (影响范围) 与 Subject (简述)

- **Scope (选填)：**标明改动影响的具体技术栈或业务模块（如：wpf, core, api）。
- **Subject (必填)：**动宾结构，精炼描述改动内容（不超过 50 字）。如果是修复 Bug，最好能一句话说清修复了什么现象，避免笼统的描述。

注意 (新手避坑)：

1. **拒绝“一镜到底”式提交：**严禁积攒数天的代码后，提交一个包含数十个文件改动的 Commit（如 feat: 完成本周工作）。应按逻辑节点小粒度提交。
2. **拒绝无意义描述：**严禁使用 fix: bug, update: 111 这种毫无意义的 Subject。
3. **严禁张冠李戴：**严格对齐上述 Type 定义，不要把重构逻辑写成了 style，或者把增加新功能写成了 refactor。

3. GitLab Flow 最佳实践 (核心协作原则)

3.1 坚持“上游优先” (Upstream First)

严禁在下游分支（`stable/vX.X` 或 `custom/xxx/vX.X`）直接“打补丁”。

如果客户报告了一个线上 1.0 版本的 Bug：

- **正确路径：**从 `main` 切出 `fix` 分支 -> 修复并合并回 `main` -> 切换到 `stable/v1.0` -> `git cherry-pick <commit-hash>` -> 打包发版。
- **错误路径：**直接在 `stable/v1.0` 修复 -> 发版 -> 忘记同步回主干 -> 2.0 版本该 Bug 再次爆发（回归错误）。

3.2 黄金法则：Rebase 优于 Merge (保持提交线笔直)

当你准备发起合并请求，发现远端 `main` 已经有了同事的新提交时，**强推使用 Rebase 同步主干，而非 Merge**。

为什么不用 Merge？`git merge main` 会在你的特性分支中产生一个无意义的合并节点（Merge branch 'main' into feat/...），导致 Git 提交网络图变得像蜘蛛网一样错综复杂。

使用 Rebase 的正确姿势：

Rebase 会将你的提交“拔起”，平移到最新主干的顶部。这样在最终合并入 `main` 时，历史记录是一条清爽的直线。

```
git checkout main
git pull origin main
git checkout feat/your-feature
git rebase main
# 如果有冲突，解决冲突后执行 git rebase --continue
git push -f origin feat/your-feature
```

注意 (新手避坑)：

1. **Rebase 的绝对禁忌：**永远不要在公共分支（如 `main`，`stable/vX.X`）上执行 Rebase，这会重写远端历史导致全员本地崩溃。**Rebase 只能在你的本地私有开发分支上使用。**
2. **冲突处理切勿 Commit：**Rebase 遇到冲突时，解决冲突后执行 `git add`，然后直接执行 `git rebase --continue`。**千万不要去执行 Commit，否则会打断变基流程。**

3.3 Squash 压缩无意义提交

在开发过程中提交的诸如 `update`，`fix typo`，`test UI` 等中间过程，在发起合并请求并合并入 `main` 时，务必开启平台上的 **Squash Commits (压缩提交)** 选项，将其合并为一个符合上述 Type 规范的干净 Commit。

4. 版本发布与标签 (Tag) 管理

标签是项目历史中的里程碑，用于精确定位每个发版的源代码快照。

4.1 语义化版本控制 (Semantic Versioning)

采用 `v主版本号.次版本号.修订号` 格式：

- **主版本 (Major)** (如 v2.0.0)：重大架构升级，API 不向下兼容。
- **次版本 (Minor)** (如 v1.5.0)：向下兼容的功能追加。
- **修订号 (Patch)** (如 v1.5.1)：向下兼容的紧急缺陷修复。

4.2 发版打标签流程

确保发布分支代码已准备就绪时，在本地创建附注标签 (Annotated Tag)：

```
git checkout stable/v1.5
git pull origin stable/v1.5
# 附注标签包含打标签的人和和时间信息，适合发版记录
git tag -a v1.5.0 -m "Release v1.5.0: 包含最新的三维视图性能优化及内存泄漏修复"
git push origin v1.5.0
```

注意 (新手避坑)： 打 Tag 前**务必先** `git pull` **最新的发布分支代码**，防止将 Tag 打在落后的历史节点上，导致发出去的包漏掉最新的修复。发版严禁使用不带描述的轻量标签 (`git tag v1.x`)。

5. 冲突解决标准作业程序 (SOP)

工程文件（如 .csproj）或复杂的 UI 布局文件（如 .xaml）发生冲突极其常见。

5.1 本地解决原则

发现冲突时，绝不在网页端强行覆盖，必须拉取到本地处理，以确保工程编译不被破坏。

5.2 冲突处理指南

- **使用专业工具：** 使用 TortoiseGit、Visual Studio 或 IDE 内置的三向对比视图（Three-way Merge）解决冲突。
- **XML/配置类文件：** 处理 .csproj 或包管理文件冲突时极度危险。不要简单地“接受当前”或“接受传入”，必须仔细检查 <ItemGroup> 或引用的类库版本，通常两方的追加内容都需要合并保留。
- **强制本地验证：** 冲突代码合并保存后，必须在本地进行一次完整的 Clean 和 Rebuild。若涉及 UI 变动，还需启动调试跑通相关流程，确认无误后才能执行 Commit 或 Rebase Continue。

注意 (新手避坑)： 很多人在代码视图里解决完冲突，保存提交就直接推送到远端。**冲突解决完不编译是绝对的红线！** 有时语法解决了，但逻辑变量名对不上，盲目推送会导致远端流水线直接挂掉。

6. 代码审查 (Code Review) 检查清单

Reviewer 必须严格把关，杜绝“纯点赞式”审查，防范潜在隐患合入主干。

6.1 代码规范与代码风格 (Code Style) (补充检查项)

- **命名规范：** 变量、方法、类名是否符合项目 C# 命名规范？命名是否做到了“见名知意”，杜绝 `var a`, `string str` 等随意命名？
- **魔法数字/字符串：** 代码中是否散落着硬编码的业务常量？（应提取为 `const` 或枚举 `Enum`）。

- **代码复杂度**：单个方法是否过长？嵌套的 `if-else` 或 `for` 循环是否过深（超过 3 层）？是否已通过“卫语句 (Guard Clauses)”提前 `Return` 来简化结构？
- **废弃代码**：提交中是否夹带了被大段注释掉的旧代码或无用的 `Console.WriteLine` 调试日志？（Git 本身即是历史，业务代码中不留死尸）。

6.2 架构与逻辑合规性

- **单一职责**：该 MR 是否只做了一件事？如果同时包含了“新增功能 (feat)”和“重构底层 (refactor)”，必须打回并要求拆分为两个独立的合并请求。
- **资源泄漏防范**：对于频繁与非托管代码交互的模块（如调用 C++ 库、COM 接口），是否正确使用了 `using` 语句释放资源？是否实现了 `IDisposable` 接口防范底层句柄泄漏？
- **UI 线程安全**：在多线程计算或异步操作结束后更新界面时，是否正确使用了跨线程调度（如 `Dispatcher`）防范 UI 崩溃？

6.3 健壮性与可维护性

- **异常捕获**：严禁直接吞掉异常（`catch (Exception) { }`）。是否针对具体的异常类型进行了捕获并记录系统日志？
- **防御性编程**：传入底层 API 的集合、指针、对象在使用前，是否进行了判空（`Null`）校验或越界检查？

7. TortoiseGit 可视化操作指南

对于习惯图形化界面的 Windows 开发者，以下将核心命令行操作映射为 TortoiseGit（小乌龟）的 UI 操作步骤。

7.1 日常开发工作流

1. 同步主干与新建分支：

- 资源管理器中右键 -> **TortoiseGit -> Switch/Checkout (切换/检出)**，选择 `main`。
- 右键 -> **Git Pull (拉取)** 获取主干最新代码。
- 右键 -> **TortoiseGit -> Create Branch (创建分支)**，命名为 `feat/xxx`，勾选 **Switch to new branch (切换到新分支)**，点击 OK。

2. 规范化提交 (Commit)：

- 右键 -> **Git Commit -> "feat/xxx"**。
- 在 Message 框中，严格按照第二章的 `<type>(<scope>): <subject>` 规范填写。
- 仔细核对下方的 **Changes (变更文件) 列表**，取消勾选自动生成的临时文件（如 `.suo`，`bin/` 目录中的文件），点击 Commit。

3. 推送 (Push)：

- 右键 -> **TortoiseGit -> Push**，点击 OK。随后前往网页端发起 MR/PR。

注意 (新手避坑)：在点击 Commit 前，一定要仔细向下滚动检查 **Changes 列表**！极易误将编译生成的 `bin/`，`obj/` 文件夹，或本地用于测试的庞大模型文件（`.dwg`，`.db` 等）勾选提交入库。这会瞬间导致 Git 仓库体积暴涨。

7.2 可视化执行 Rebase (变基)

1. **拉取主干最新指针**：右键 -> **TortoiseGit -> Fetch (获取)**，将远端更新抓取到本地仓库。
2. **启动变基**：在当前开发分支上右键 -> **TortoiseGit -> Rebase (变基)**。
3. **设置上游**：在 **Upstream (上游)** 选框中，选择 `remotes/origin/main`。

4. 执行并强制推送：点击 **Start Rebase**。变基完成后，执行 Push 时需勾选 **"Force Known Changes" (强制推送)**。

注意 (新手避坑)： TortoiseGit 里的 "Force Known Changes"（强制推送）是一个极度危险的操作。只有在推送你自己的、刚刚经过 Rebase 的个人开发分支时才能使用！绝不允许勾选该项后推送到 main 或 stable/vX.X。

7.3 可视化合并无意义提交 (Squash)

1. 右键 -> **TortoiseGit -> Rebase (变基)**。
2. 勾选下方的 **Force Rebase** 以显示当前分支的所有提交历史。
3. 在列表中，保留最底部的基础提交状态为 **Pick**。
4. 选中上方所有需要压缩的中间过程提交（如 `fix typo`，`test`），右键 -> **Squash (压缩)**。
5. 点击 **Start Rebase**，在弹窗中重新编写一个符合规范的整合 Commit Message，保存即可。

7.4 图形化解决冲突 (TortoiseMerge)

当发生合并冲突时，冲突文件会被标记红色感叹号：

1. 右键冲突文件 -> **TortoiseGit -> Edit Conflicts (编辑冲突)**。
2. 在弹出的三向对比视图中（左侧 **Theirs** 远端 / 右侧 **Mine** 本地 / 下方 **Merged** 结果），右键点击冲突高亮块，选择保留哪一方的代码（或两者组合）。
3. 确认下方面板的最终代码逻辑正确后，保存修改并关闭窗口。
4. 右键该文件 -> **TortoiseGit -> Resolve (标记为已解决)**。如果是 Rebase 过程中的冲突，返回 Rebase 窗口并点击 **Continue** 继续变基。